Tribhuvan University

Faculty of Humanities and Social Sciences

Masters of Computer Application (MCA)

1st Semester

Advanced Database Management System(MCA-503)

Unit-3: Query Optimization 9hrs

Instructor

Tekendra Nath Yogi (M.SC CSIT TU)

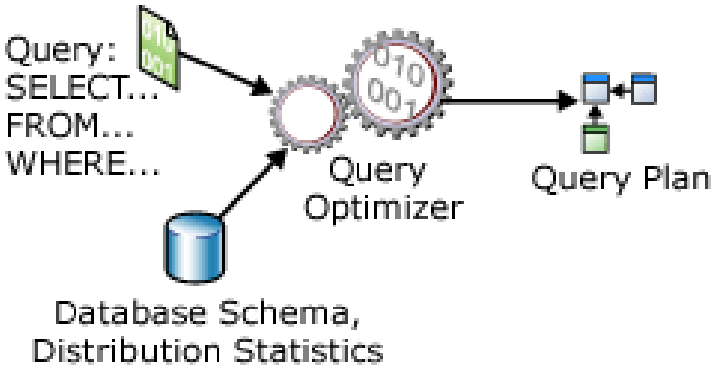
Tekendranath@gmail.com

Contents

- **Unit 3: Query Optimization [9 Hrs.]**
  - 3.1. Query Trees and Heuristics for Query Optimization;
  - 3.2. Choice of Query Execution Plans;
  - 3.3. Use of Selectivities in cost base optimization;
  - 3.4. Cost Functions for SELECT operation;
  - 3.5. Cost Functions for the JOIN Operations;
  - 3.6. Example to illustrate Cost Based Query Optimization;
  - 3.7. Additional Issues Related to Query Optimization;
  - 3.8. An Example of Query Optimization in Data warehouses

Introduction to Query Optimization

- **Query optimization** is a activity conducted by a query optimizer in a DBMS to select best available strategy for executing query.
  - The goal of query optimization is to select the best possible strategy for query evaluation.
  - i.e., Arrive at the most **efficient** and **cost-effective plan** in a reasonable amount of time using the available information about the schema and the content of relations involved.
  - However, the chosen execution plan may not always be the most optimal plan possible.

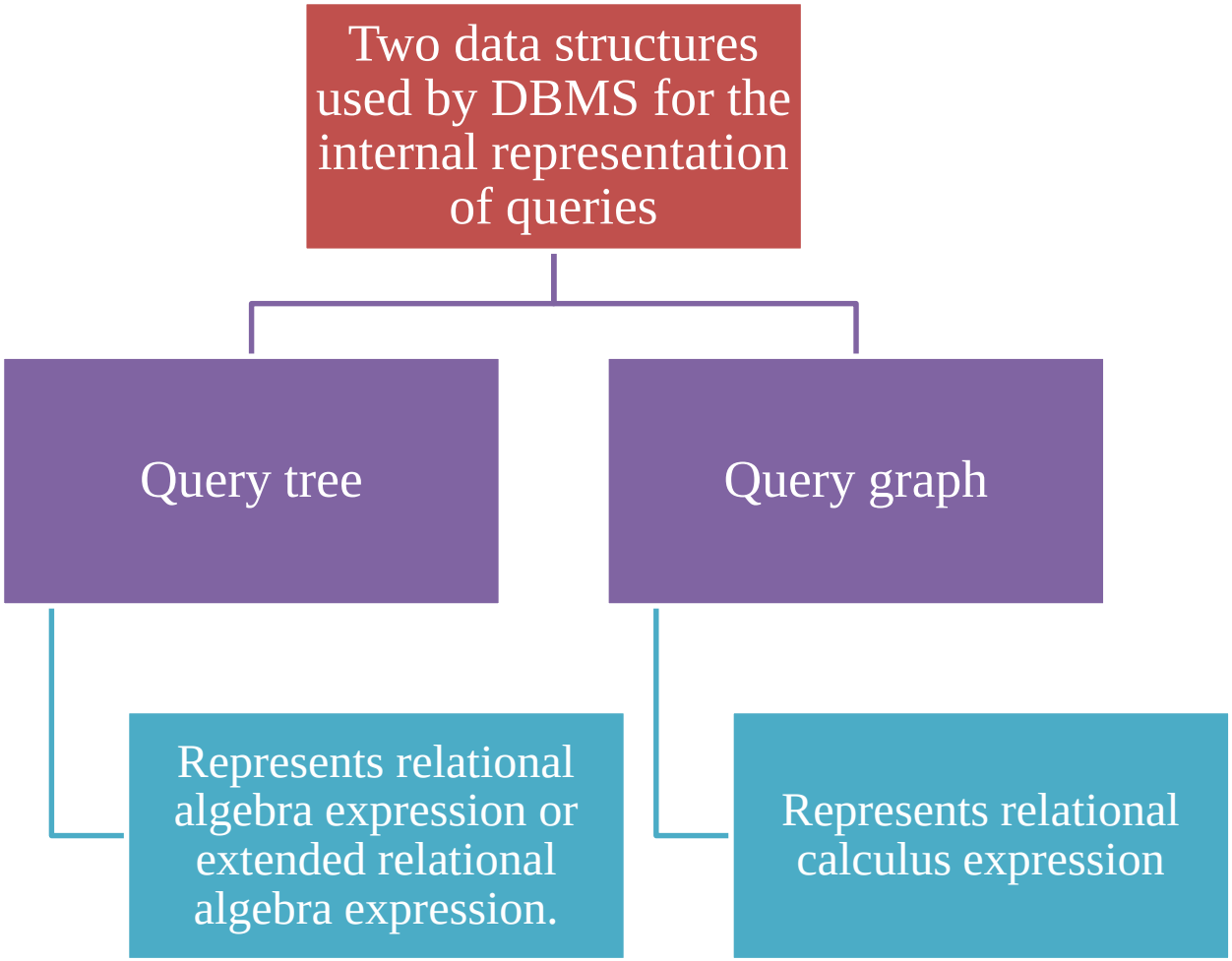


Cont'd...

- There are two main techniques of query optimization:
  - **Heuristic Query Optimization**: Based on **heuristic** rules for ordering the operations in a query execution strategy.
    - i.e., Applies heuristic rules to modify the internal representation of a query to improve its expected performance.
    - Example heuristic rule: Apply SELECT and PROJECT before JOIN that Reduces size of files to be joined
  - **Cost Based Query Optimization**: Estimate the cost of different execution strategies and choose the execution plan that minimizes estimated cost.

3.1. Query Trees and Heuristics for Query Optimization

- **3.1.1. Notation for Query Trees and Query Graphs:**



Note: Most RDBMSs use a tree as the internal representation of a query

Cont'd…

- **Query Tree:** A query tree is a tree data structure that corresponds to an extended relational algebra expression.
  - It represents the **input relations** of the query as **leaf nodes** of the tree, and it represents the **relational algebra operations** as **internal nodes**.

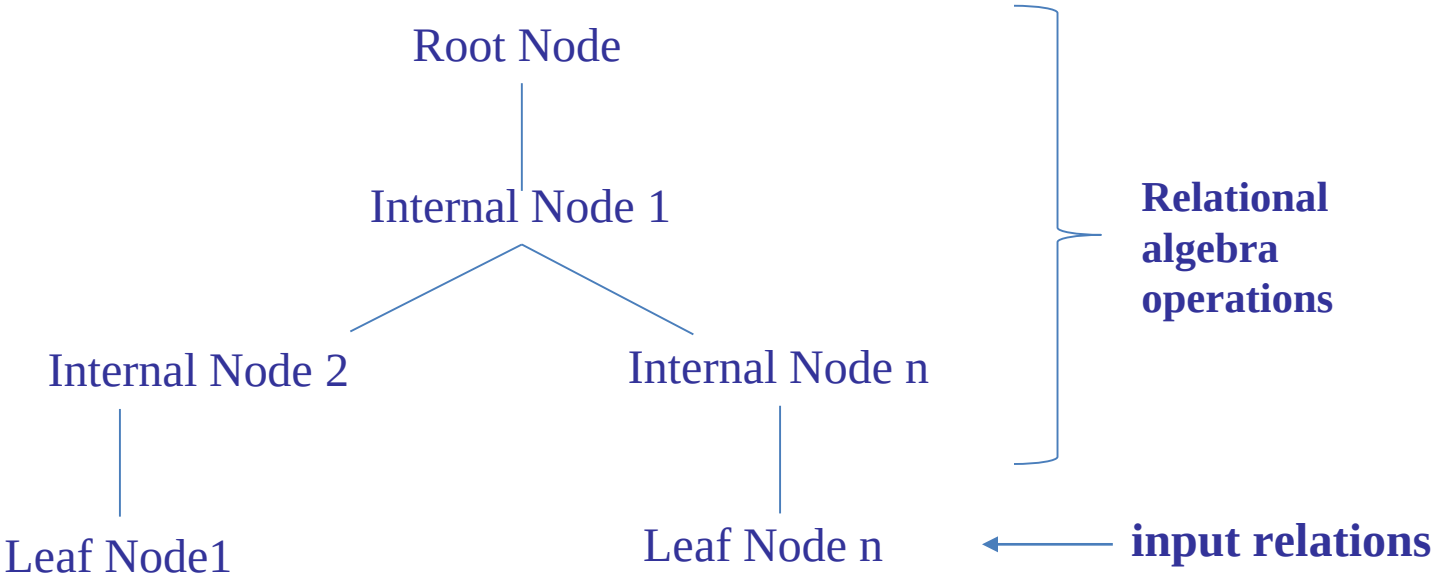


Figure: General Structure of query tree

Cont'd…

- **Execution of the Query Tree:**
  - An execution of the query tree consists of executing an internal node operation whenever its operands are available and then replacing that internal node by the relation that results from executing the operation.
  - The execution of operations starts at the leaf nodes, which represents the input database relations for the query, and ends at the root node, which represents the final operation of the query.
  - The execution terminates when the root node operation is executed and produces the result relation for the query.

Cont'd…

- **Execution of the Query Tree:**

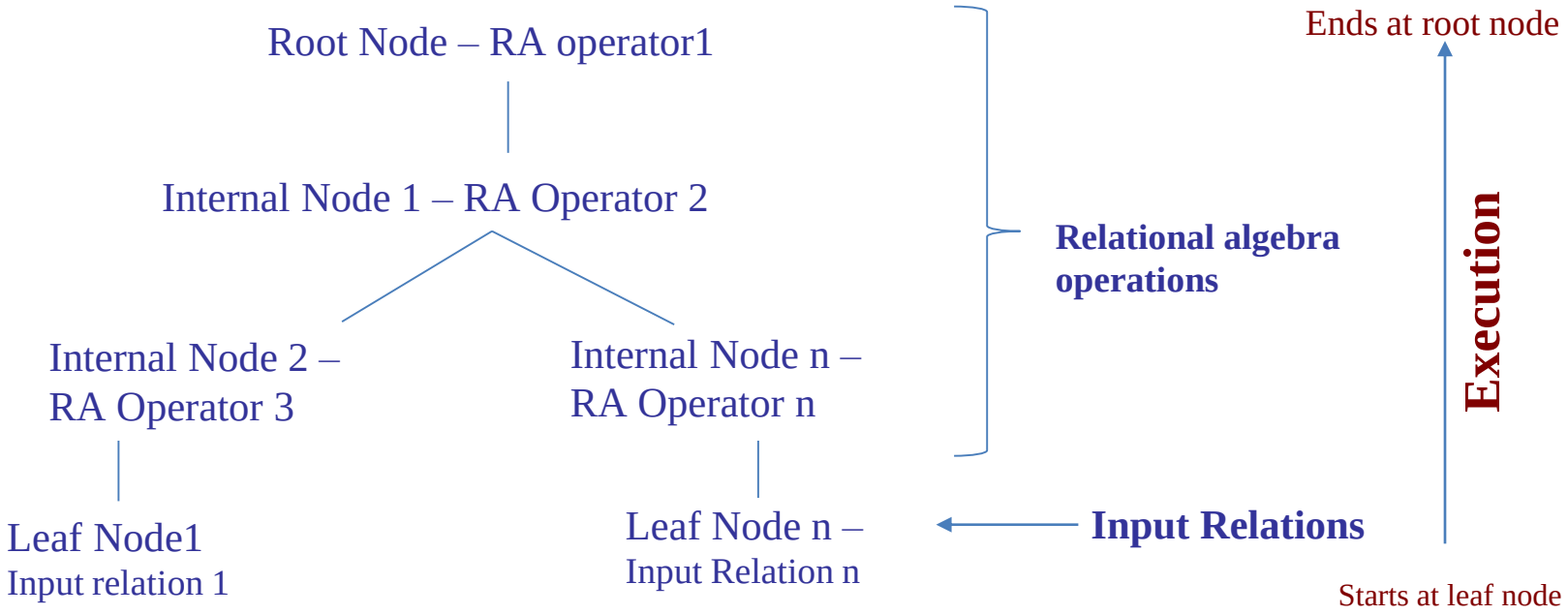


Figure: Execution order of the query in the query tree

- Query trees are preferred over query graphs because they explicitly represent the specific order of operations required for query execution, which is essential for query optimizers.

Cont'd…

- **Example**: For every project located in 'Stafford', retrieve the project number, the controlling department number and the department manager's last name, address and birthdate.
- SQL query Q2:

```
SELECT    P.NUMBER, P.DNUM, E.LNAME, E.ADDRESS, E.BDATE
FROM      PROJECT AS P, DEPARTMENT AS D, EMPLOYEE AS E
WHERE     P.DNUM=D.DNUMBER AND D.MGR_SSN=E.SSN AND P.PLOCATION='STAFFORD';
```

- Relation algebra:

$$\pi_{P.PNUMBER,P.DNUM,\ E.LNAME,\ E.ADDRESS,\ E.BDATE}\ (((\sigma_{P.PLOCATION='STAFFORD'}(PROJECT)) \bowtie_{P.DNUM=D.DNUMBER} (DEPARTMENT)) \bowtie_{D.MGR\_SSN=SE.SN} (EMPLOYEE))$$

Cont'd…

- The equivalent Query tree for the above relational algebra for the given query is as shown in figure below:

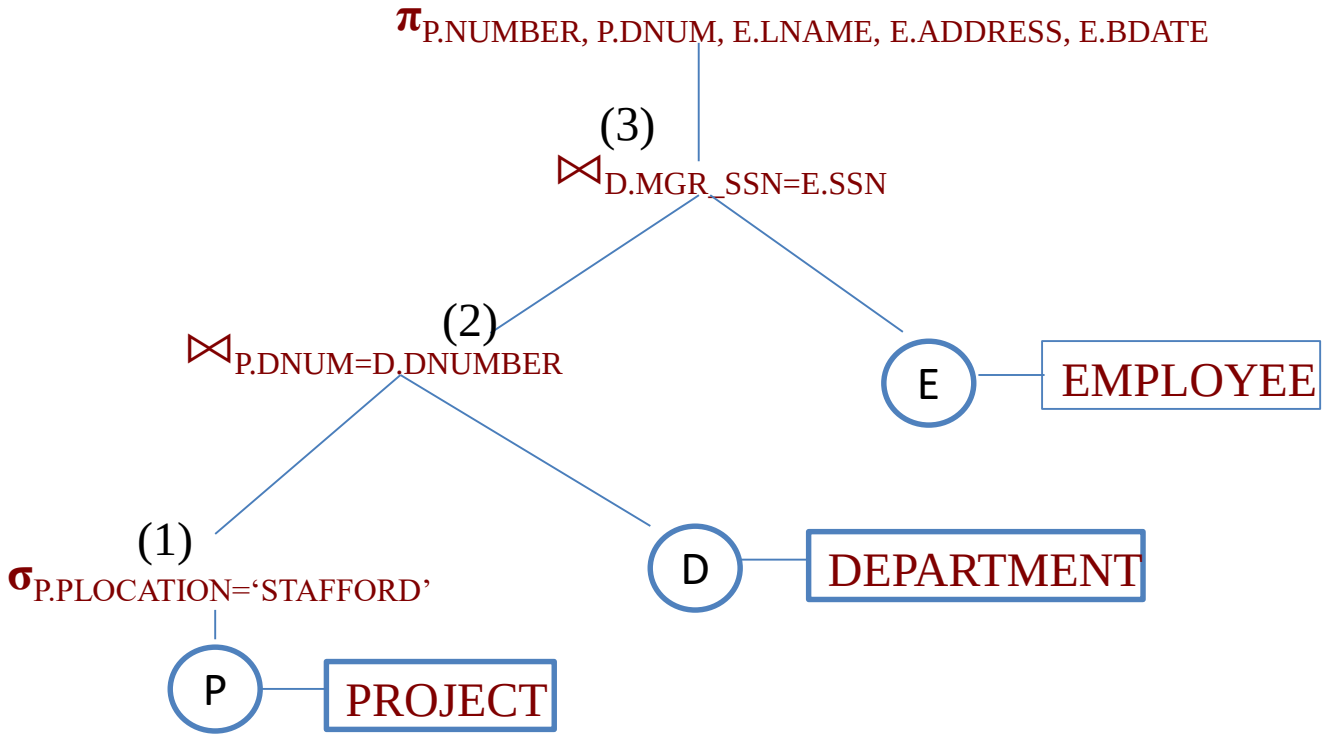


Figure: Query tree corresponding to the relational algebra expression for Q2.

the leaf nodes P, D, and E represent the three relations PROJECT, DEPARTMENT, and EMPLOYEE, respectively. Internal nodes: Relational algebra operations Order of Execution: node (1) -> (2) -> and then (3) and so on.

Cont'd…

- **Query graph:**
  - Represents relational calculus expression
  - Relation nodes displayed as single circles
  - Constants represented by Double circles or ovals
  - Selection or join conditions represented as edges
  - Attributes to be retrieved displayed in square brackets above each relation.
  - For example,

Figure: Query graph for Q2.

Cont'd…

- **Query graph:**
  - There is only a single graph corresponding to each query.
  - The query graph representation does not indicate an order on which operations to perform first.
  - In practice, the query optimizer needs to show the order of operations for query execution, which is not possible in query graphs.
  - Hence, Query graph representation is rarely used by the DBMSs in the internal representation of the query.

Cont'd…

- **3.1.2. Heuristic Optimization of Query Trees:** Heuristic query optimization involves the following steps:
  - **Step 1:** Scanner and Parser generate initial query representation (Query Tree or graph)
  - **Step 2:** Representation is optimized according to heuristic rules: are applied to convert an initial query tree into an equivalent query tree, which represents a different relational algebra expression that is more efficient to execute but gives the same result as the original tree.
  - **Step 3:** Query execution plan is then generated from the optimized query representation to execute groups of operations based on access paths available and files involved in the query.

Cont'd…

- **Query Transformation Example:** Consider the following query to Find the last names of employees born after 1957 who work on a project named 'Aquarius'.:

```
Q: SELECT    E.LNAME
   FROM      EMPLOYEE E, WORKS_ON W, PROJECT P
   WHERE     AND P.PNMUBER=W.PNO
             AND E.ESSN=W.SSN
             AND E.BDATE > '1957-12-31'
             AND P.PNAME = 'AQUARIUS' ;
```

- Equivalent Relational algebraic expression is:
  - $\pi_{E.LNAME}(\sigma_{P.PNUMBER=W.PNO \wedge E.ESSN=W.SSN \wedge E.BDATE>'1957-12-31' \wedge P.PNAME='AQUARIUS'}(EMPLOYEE \times WORKS\_ON \times PROJECT))$

Cont'd…

- The initial query tree for Q is shown in Figure below:

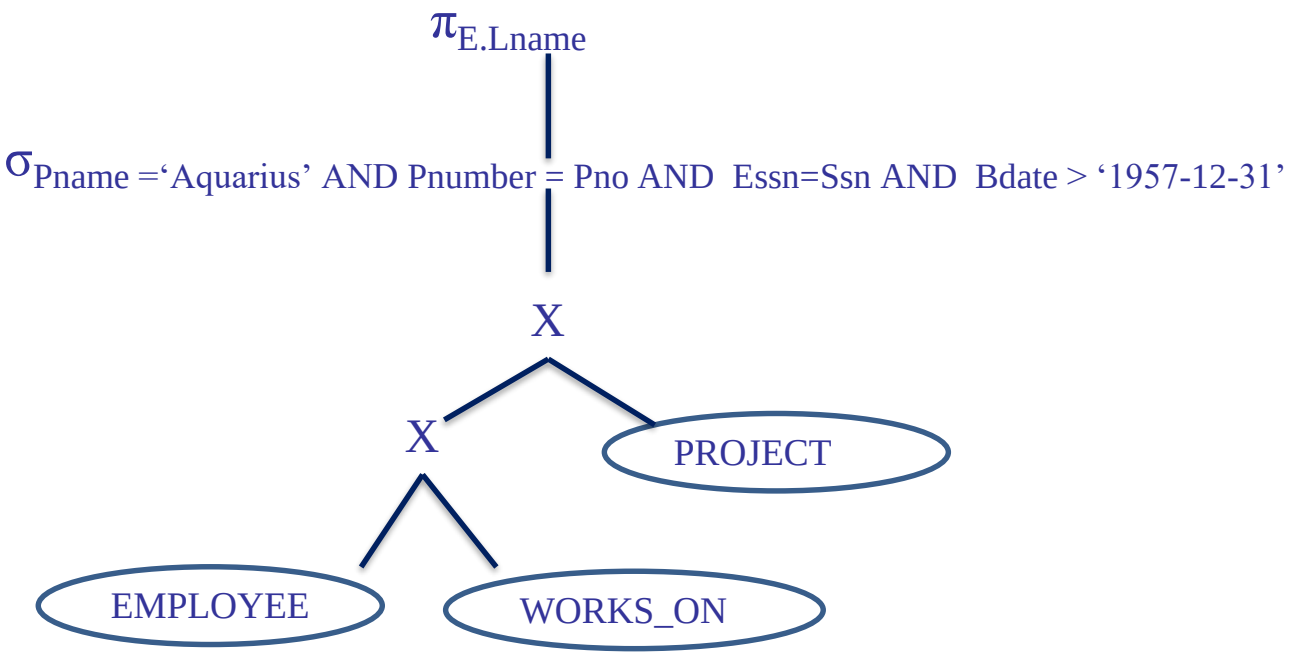


Figure: Initial (canonical) query tree for SQL query Q.

Executing this tree directly first creates a very large file containing the CARTESIAN PRODUCT of the entire EMPLOYEE, WORKS_ON, and PROJECT files. That is why the initial query tree is never executed, but is transformed into another equivalent tree that is efficient to execute.

Cont'd…

- Figure below shows an improved query tree that first applies the SELECT operations to reduce the number of tuples that appear in the CARTESIAN PRODUCT.

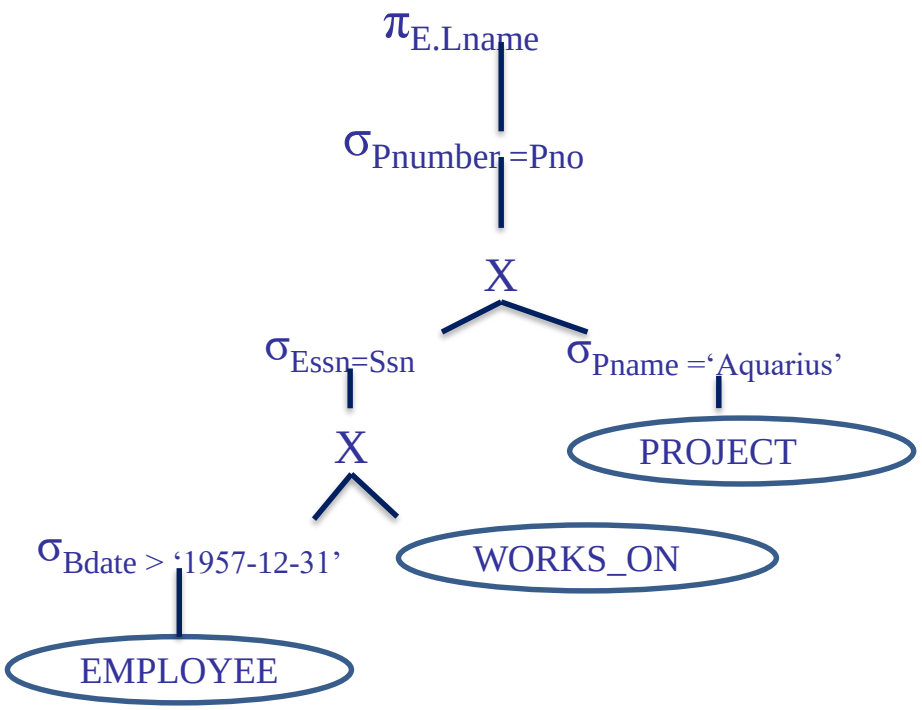


Figure: Moving SELECT operations down the query tree.

Cont'd…

- A further improvement is achieved by switching the positions of the EMPLOYEE and PROJECT relations in the tree, as shown in Figure below:

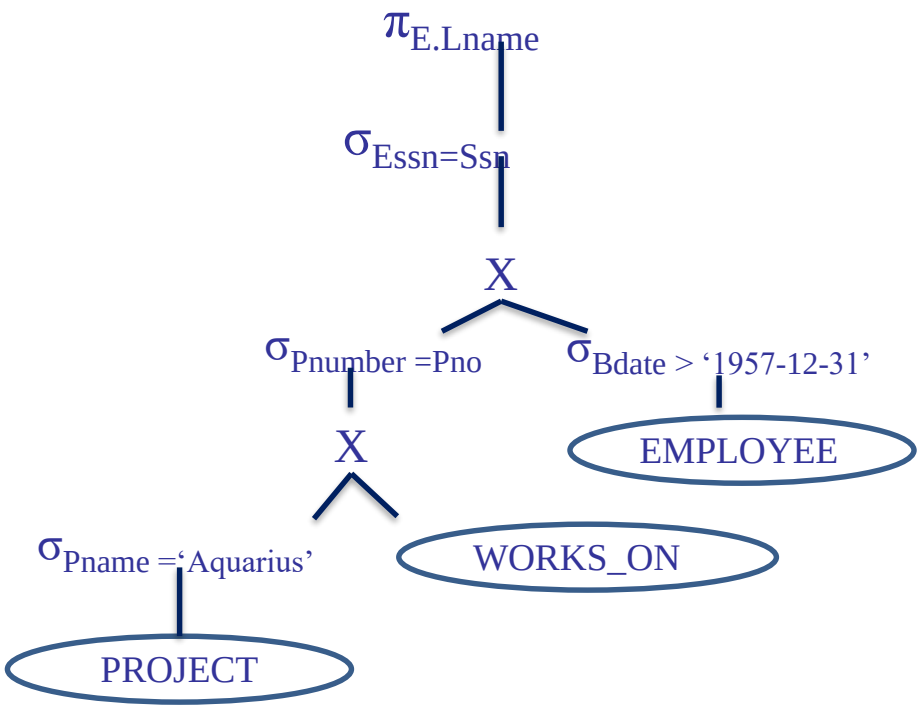


Figure: Applying the more restrictive SELECT operation first.

Cont'd…

- Further improve the query tree by replacing any CARTESIAN PRODUCT operation that is followed by a join condition as a selection with a JOIN operation, as shown in Figure below.

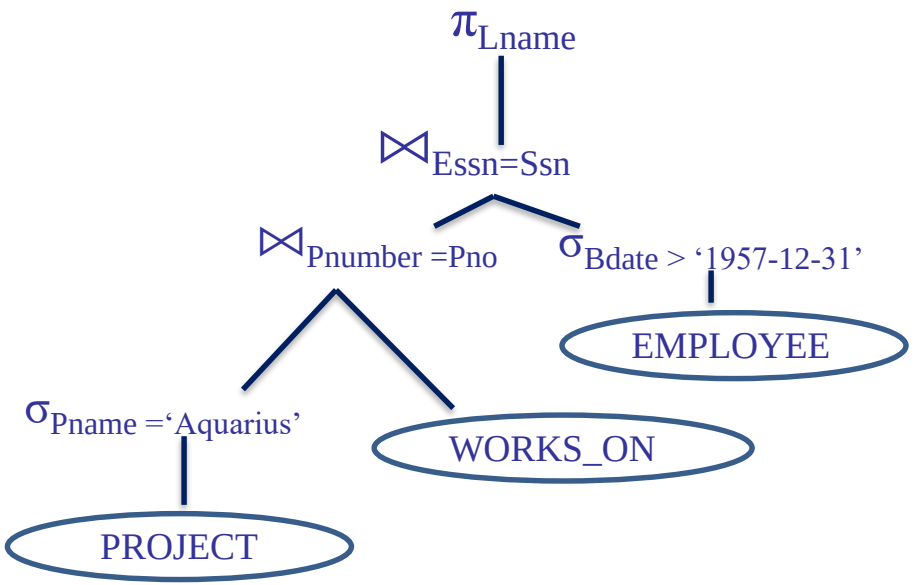


Figure: Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.

Cont'd…

- Another improvement is to keep only the attributes needed by subsequent operations in the intermediate relations, by including PROJECT (π) operations as early as possible in the query tree, as shown in Figure below:

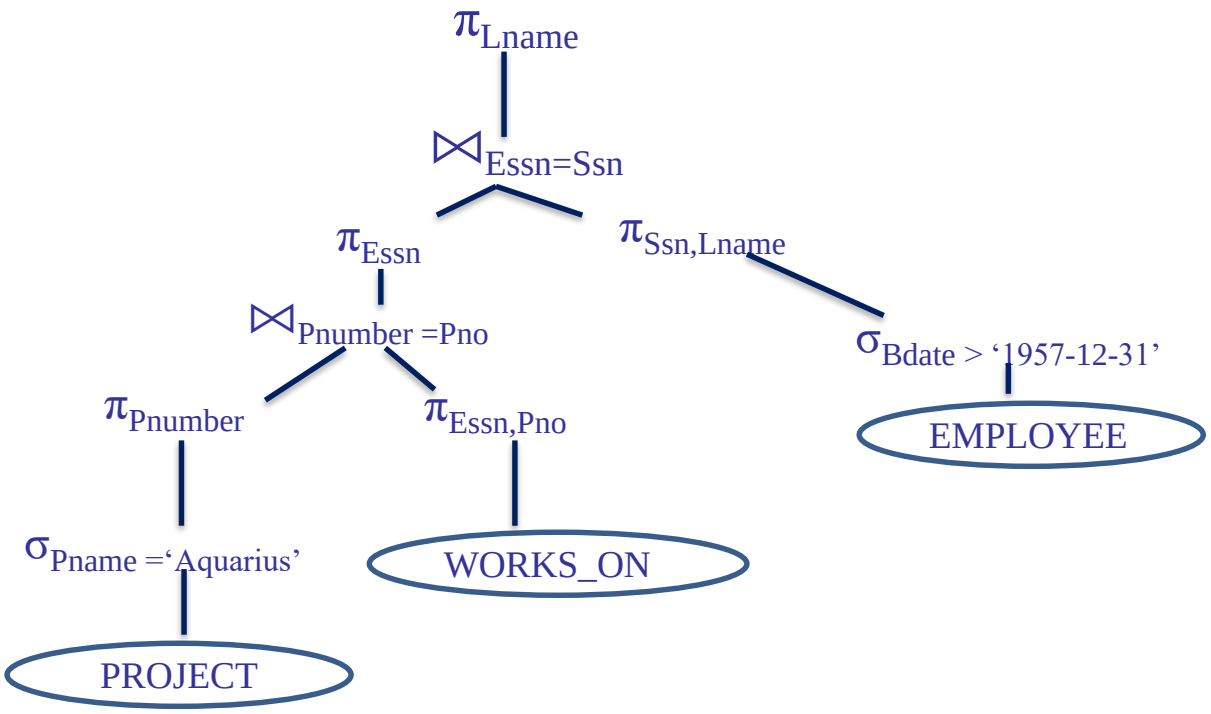


Figure: Moving PROJECT operations down the query tree.

In this way a query optimizer transform the initial query tree into an equivalent optimized query tree that is more efficient to execute.

Cont'd…

- **3.1.3. General Transformation Rules for Rational Algebra Operations:**

  Some transformation rules useful in query optimization

  1. **Cascade of σ:** A conjunctive selection condition can be broken up into a cascade (sequence) of individual σ operations:

     - $\sigma_{c1\ AND\ c2\ AND\ ...\ AND\ cn}(R) = \sigma_{c1}(\sigma_{c2}(...(\sigma_{cn}(R))...))$

  2. **Commutativity of σ**: The σ operation is commutative:

     - $\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R))$

Cont'd…

3. **Cascade of π**: In a cascade (sequence) of π operations, all but the last one can be ignored:
   - $\pi_{List1}(\pi_{List2}(...(\pi_{Listn}(R))...)) = \pi_{List1}(R)$
4. **Commuting σ with π**: If the selection condition c involves only the attributes A1, ..., An in the projection list, the two operations can be commuted:
   - $\pi_{A1, A2, ..., An}(\sigma_c(R)) = \sigma_c(\pi_{A1, A2, ..., An}(R))$
5. **Commutativity of $\bowtie$ (and x):** The $\bowtie$ operation is commutative as is the x operation:
   - $R \bowtie_C S = S \bowtie_C R$;
   - R x S = S x R

Cont'd…

6. Commuting σ with ⋈ (or x):

- If all the attributes in the selection condition c involve only the attributes of one of the relations being joined—say, R—the two operations can be commuted as follows:
  - $\sigma_c (R \bowtie S) = (\sigma_c (R)) \bowtie S$
- Alternatively, if the selection condition c can be written as (c1 and c2), where condition c1 involves only the attributes of R and condition c2 involves only the attributes of S, the operations commute as follows:
  - $\sigma_c (R \bowtie S) = (\sigma_{c1} (R)) \bowtie (\sigma_{c2} (S))$

Cont'd…

7. **Commuting π with ⋈ (or x):**

- Suppose that the projection list is L = {A_1, ..., A_n, B_1, ..., B_m}, where A_1, ..., A_n are attributes of R and B_1, ..., B_m are attributes of S. If the join condition c involves only attributes in L, the two operations can be commuted as follows:
  - $\pi_L (R \bowtie_C S) = (\pi_{A1, ..., An} (R)) \bowtie_C (\pi_{B1, ..., Bm} (S))$
- If the join condition C contains additional attributes not in L, these must be added to the projection list, and a final π operation is needed.

- **8. Commutativity of set operations**: The set operations υ and ∩ are commutative but "–" is not.

Cont'd…

9. **Associativity of ⋈, x, υ, and ∩ :** These four operations are individually associative; that is, if θ stands for any one of these four operations (throughout the expression), we have
   - $(R \theta S) \theta T = R \theta (S \theta T)$
10. **Commuting σ with set operations**: The σ operation commutes with υ , ∩ , and –. If θ stands for any one of these three operations, we have
    - $\sigma_c (R \theta S) = (\sigma_c (R)) \theta (\sigma_c (S))$
- **11. The π operation commutes with υ.**

$$\pi_L (R \upsilon S) = (\pi_L (R)) \upsilon (\pi_L (S))$$

Cont'd…

- **12. Converting a (σ, x) sequence into ⋈ :** If the condition c of a σ that follows a x Corresponds to a join condition, convert the (σ, x) sequence into a ⋈ as follows:

$$(\sigma_C (R \times S)) = (R \bowtie_C S)$$

- **13. Pushing σ in conjunction with set difference.**
  - $\sigma_c (R - S) = \sigma_c (R) - \sigma_c (S)$

  However, σ may be applied to only one relation:
  - $\sigma_c (R - S) = \sigma_c (R) - S$
- **14. Pushing σ to only one argument in ∩.**
  - If in the condition σ_c all attributes are from relation R, then: $\sigma_c (R \cap S) = \sigma_c (R) \cap S$

Cont'd…

- **15. Some trivial transformations:** If S is empty, then $R \cup S = R$ as follows: If the condition c in σ_c is true for the entire R, then $\sigma_c (R) = R$.
- **De Morgan's laws:** For conditions C for selection and joins
  - NOT (c_1 AND c_2) ≡ (NOT c_1) OR (NOT c_2)
  - NOT (c_1 OR c_2) ≡ (NOT c_1) AND (NOT c_2)
- **Other transformations:** many more other transformations rules exist in the relational algebra in order to transform one algebra operation into equivalent another one.

Cont'd…

- **Heuristic Algebraic Optimization Algorithm:**

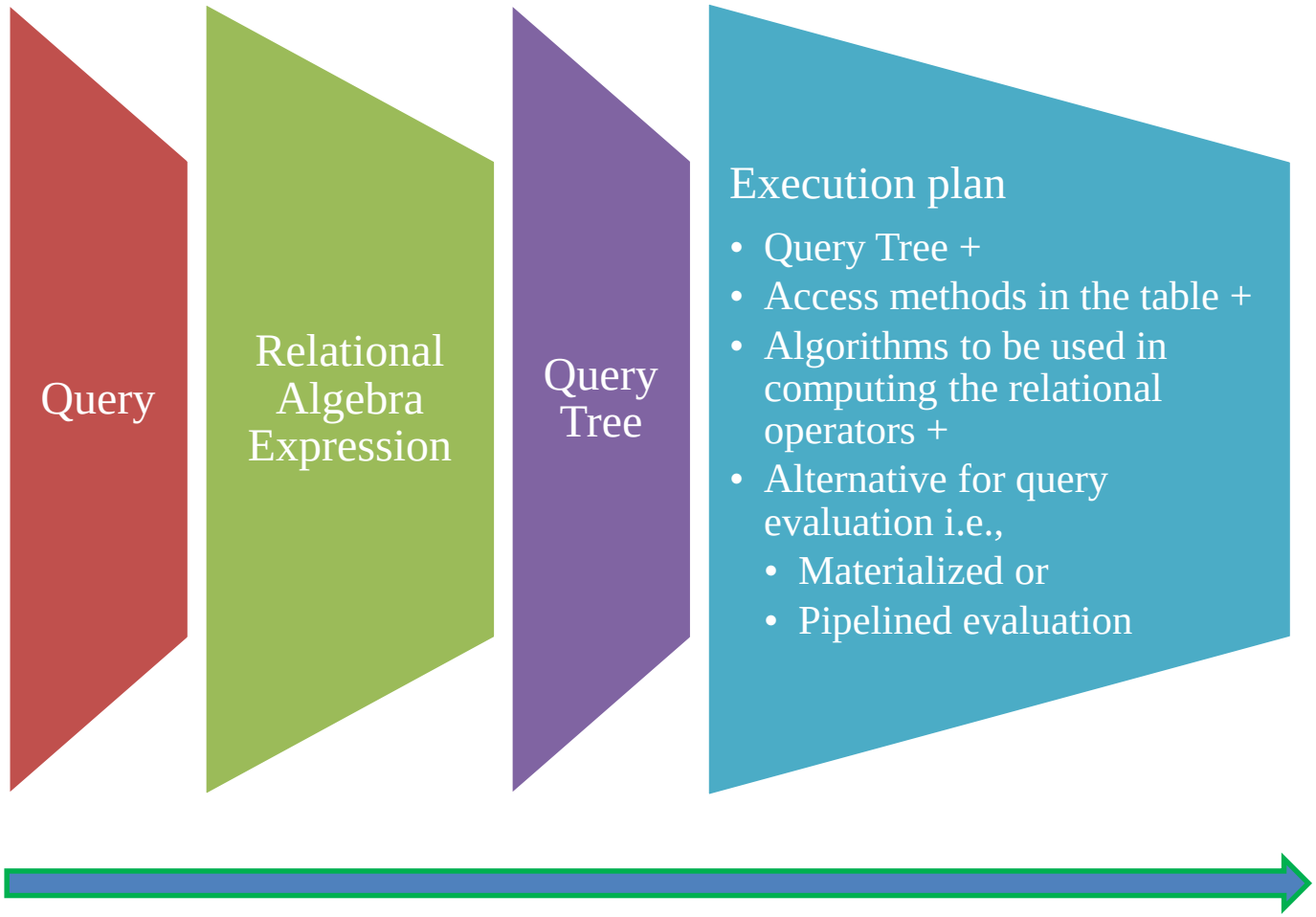
1. Using rule 1, break up any select operations with conjunctive conditions into a cascade of select operations.
2. Using rules 2, 4, 6, and 10 concerning the commutativity of select with other operations, move each select operation as far down the query tree as is permitted by the attributes involved in the select condition.
3. Using rule 9 concerning associativity of binary operations, rearrange the leaf nodes of the tree so that the leaf node relations with the most restrictive select operations are executed first in the query tree representation.
4. Using Rule 12, combine a Cartesian product operation with a subsequent select operation in the tree into a join operation.
5. Using rules 3, 4, 7, and 11 concerning the cascading of project and the commuting of project with other operations, break down and move lists of projection attributes down the tree as far as possible by creating new project operations as needed.
6. Identify subtrees that represent groups of operations that can be executed by a single algorithm.

Cont'd…

- **Summary of Heuristics for Algebraic Optimization:**
  - Apply first the operations that reduce the size of intermediate results.
  - For example,
    - Perform SELECT and PROJECT operations as early as possible to reduce the number of tuples and attributes
    - The SELECT and JOIN operations that are most restrictive should be executed before other similar operations

3.2 Choice of Query Execution Plans

- **3.2.1. Alternatives for Query Evaluation:**



Cont'd…

- Two alternatives for the query Evaluation:
  - Materialized evaluation
    - Result of an operation stored as temporary relation that is, the result is physically materialized.
  - Pipelined evaluation
    - Operation results forwarded directly to the next operation in the query sequence
    - The advantage of pipelining is the cost savings.
    - Hence, is preferred whenever feasible.

Cont'd…

- Example: From Query to Query Plan

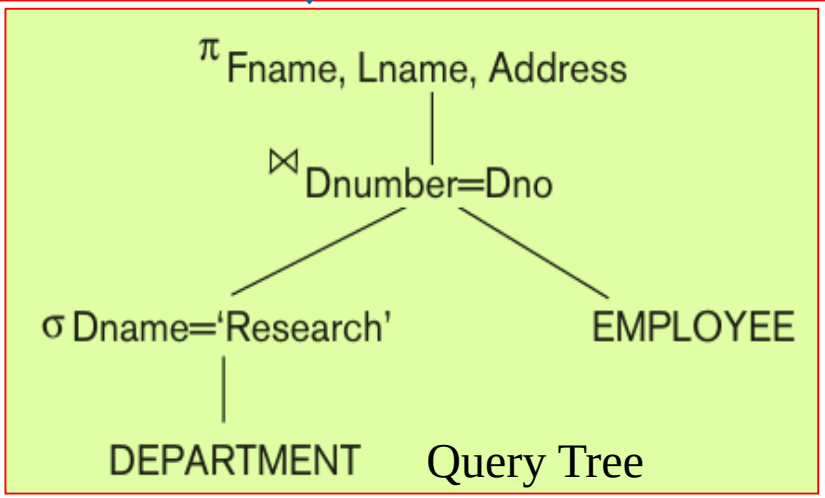
Query:
Select Fname, Lname, Address
From DEPARTMENT, EMPLOYEE
Where Dnumber=Dno AND Dname='Research

Relational Algebra: $\pi_{Fname, Lname, Address}(\sigma_{Dname='Research'}(DEPARTMENT) \bowtie_{Dnumber=Dno} EMPLOYEE)$

A table scan

Index-based nested-loop join algorithm

Index search



Query Tree

materialized or a pipelined evaluation

Figure: Query Evaluation Plan

Cont'd…

- **3.2.2. Nested Subquery Optimization**: Two types of query nesting:
  - Nested subqueries in the where clause of the SQL.
    - **Correlated**: an inner block makes a reference to the relation used in the outer block
    - **Un-Correlated**: an inner block do not makes a reference to the relation used in the outer block
  - Nested subqueries in the From clause of the SQL.
- The naïve strategy for evaluating the query is to evaluate the inner nested subquery for every tuple of the outer relation, which is inefficient.
- Hence, query optimizer need to transform nested subqueries and rewrites the entire query to make the efficient evaluation of them.

Cont'd…

- Two variations of query transformation and rewriting used by the query optimizer (transforms them and rewrites the entire query) in order optimize the nested subqueries.
- These are:
  - Nested subquery optimization (Unnesting or Decorrelation) and
  - Subquery (view) merging transformation.

Cont'd…

- **Nested subquery optimization:** Wherever possible, SQL optimizer tries to convert queries with nested subqueries into a join operation.
- Following alternatives :
  - Correlated nested subqueries are un-nested into a join operation.
  - Subqueries nested using EXISTS, IN, and ANY un-nested using Semi-join
  - Subqueries nested using NOT IN, NOT EXISTS and ALL un-nested using Anti-join
- The join can then be evaluated with one of the algorithm used for the join operation.

Cont'd…

- **Example: Join**

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE E WHERE EXISTS (
        SELECT *
        FROM DEPARTMENT D
        WHERE D.Dnumber = E.Dno AND D.Zipcode=30332);
```

Unnesting

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE E, DEPARTMENT D
WHERE D.Dnumber = E.Dno AND D.Zipcode=30332
```

Cont'd...

- **Example**: **Semi-join:**

```
SELECT COUNT(*)
FROM DEPARTMENT D
WHERE D.Dnumber IN (
                    SELECT E.Dno
                    FROM EMPLOYEE E
                    WHERE E.Salary > 200000
                   );
```

Un-nesting

```
SELECT COUNT(*)
FROM DEPARTMENT D, EMPLOYEE E
WHERE D.Dnumber S= E.Dno  AND E.Salary > 200000;
```

Query counts the number of departments that have employees who make more than 200,000 annually.

Cont'd...

- **Example: Anti-join Example**

```
SELECT COUNT(*)
FROM   EMPLOYEE
WHERE  EMPLOYEE.Dno NOT IN
        (
                SELECT DEPARTMENT.Dnumber
               FROM   DEPARTMENT
                WHERE  Zipcode = 30332
        );
```


Un-nesting

```
SELECT COUNT(*)
FROM   EMPLOYEE, DEPARTMENT
WHERE  EMPLOYEE.Dno A= DEPARTMENT.Dnumber
AND    Zipcode = 30332;
```

Query counts the number of employees who do not work in departments located in zip code 30332.

Cont'd…

- **3.2.3. Subquery (View) Merging Transformation**:
- A subquery may appears in the FROM clause of a query .
- This FROM clause subquery is often referred to as an inline View.
- The transformation of the such subquery referred to as a view-merging or subquery merging transformation.
- View merging operation:
  - Merges the tables in the view with the tables from the outer query block
  - Views containing select-project-join operations are considered simple views
    - Can always be subjected to this type of view-merging

Cont'd…

- **Example:**

```
SELECT E.Ln, V.Addr, V.Phone
FROM EMP E, (
        SELECT D.Dno, D.Dname, B.Addr, B.Phone
        FROM DEPT D, BLDG B WHERE D.Bldg_id = B.Bldg_id ) V
WHERE V.Dno = E.Dno AND E.Fn = "John";
```

View-Merge

```
SELECT E.Ln, B.Addr, B.Phone
FROM EMP E, DEPT D, BLDG B
WHERE D.Bldg_id = B.Bldg_id AND D.Dno = E.Dno AND E.Fn =
"John";
```

Cont'd…

- **Group-By view-merging**
  - Delaying the Group By operation after performing joins may reduce the data subjected to grouping in case the joins have low join selectivity.
  - Alternately, performing Group By early may reduce the amount of data subjected to subsequent joins.
  - Optimizer determines whether to merge GROUP-BY views based on estimated costs

Cont'd…

- **3.2.4. Materialized Views**:
  - View defined in database as a query
    - Materialized view stores results of that query (derived data)
      - May be stored temporarily or permanently
  - Materialized views as a another Optimization technique:
    - Using materialized views to avoid some of the computation involved in the query
    - Easier to read it when needed than recompute from scratch
    - Allow more queries to be processed against it
  - The savings can be significant when the:
    - View involves costly operations like join, aggregation, and so forth.
    - Information is frequently required in reports or queries

Cont'd…

- View updated (refreshed) when any of the underlying base relations have a change in the form of insert, delete, and modify.
- Three update strategies for updating the view:
  - Immediate update, which updates the view as soon as any of the relations participating in the view are updated
  - Lazy update, which recomputes the view only upon demand
  - Periodic update (or deferred update), which updates the view later, possibly with some regular frequency
- The straightforward and naive approach is to recompute the entire view for every update to any base table and is prohibitively costly.
- Hence incremental view maintenance is done in most RDBMSs today as an optimization technique for maintaining views.
- **Incremental View Maintenance**: Update view incrementally by accounting for changes that occurred since last update.

Cont'd…

- **Incremental View Maintenance**: Join
  - If a view contains inner join of relations r and s, $v_{old} = r \bowtie s$, and there is a new set of tuples inserted: r_i, in r, then the new value of the view contains $(r \cup r_i) \bowtie s$.
  - The incremental change to the view can be computed as:

    $v_{new} = r \bowtie s \cup r_i \bowtie s$

    i.e., Join only new tuples with S.
  - Similarly, deleting a set of tuples r_d from r results in the new view as:

    $v_{new} = r \bowtie s - r_d \bowtie s$.

    i.e., Remove affected join results.
  - Similar logic applies when s undergoes addition or deletion.

Cont'd…

- **Incremental View Maintenance Example** : Selection
  - If a view is defined as $V = \sigma_C R$ with condition C for selection, when a set of tuples r_i are inserted into r, the view can be modified as:

    $v_{new} = v_{old} \cup \sigma_C r_i$.

    i.e., Add to view only if it satisfies condition C.
  - On the other hand, upon deletion of tuples r_d from r, we get:

    $v_{new} = v_{old} - \sigma_C r_d$.

    i.e., Remove from view if it satisfies condition C.

Cont'd…

- **Incremental View Maintenance**: Projection
  - Assume that SELECT DISTINCT is being used in defining the view to correspond to the project (π) operation.
  - view maintenance of projection views requires a count to be maintained in addition to the actual columns in the view.
  - Each time an insert to the base relation results in contributing to the view, the count is incremented; if a deleted tuple from the base relation has been represented in the view, its count is decremented.
  - When the count of a tuple in the view reaches zero, the tuple is actually dropped from the view.
  - When a new inserted tuple contributes to the view, its count is set to 1.

Cont'd…

- **Incremental View Maintenance**: Intersection
  - If the view is defined as V= R ∩ S, when a new tuple r_i is inserted, it is compared against the s relation to see if it is present there. If present, it is inserted in v, else not.
  - If tuple r_d is deleted, it is matched against the view v and, if present there, it is removed from the view.

Cont'd…

- **Incremental View Maintenance**: Aggregation
- Suppose a materialized view is defined as:

```
SELECT G, Aggregate_Function(A)
FROM R
GROUP BY G;
```

- Where:
  - G = Grouping attribute
  - A = Attribute on which aggregation is performed
- The view stores one aggregate value for each group.
- Instead of recomputing the entire view whenever tuples are inserted or deleted, incremental view maintenance updates only the affected group.

Cont'd…

- **Incremental View Maintenance**: Count Aggregation
  - Maintain count per group as follows:
  - On insert in the base relation then increment the count in the view as:

    Count +1
  - On delete in the base relation then decrement the count in the view as:

    Count −1 and Remove group when count = 0

Cont'd…

- **Incremental View Maintenance**: Sum Aggregation
  - To maintain the sum aggregation on the view maintain the Sum and Count aggregation per group as:
  - On Insert into the base relation maintain the sum in the view as:
    - Sum=Sum + Value
    - Count = Count + 1
  - On delete into the base relation maintain the sum in the view as:
    - Sum=Sum−Value
    - Count =Count -1

Cont'd…

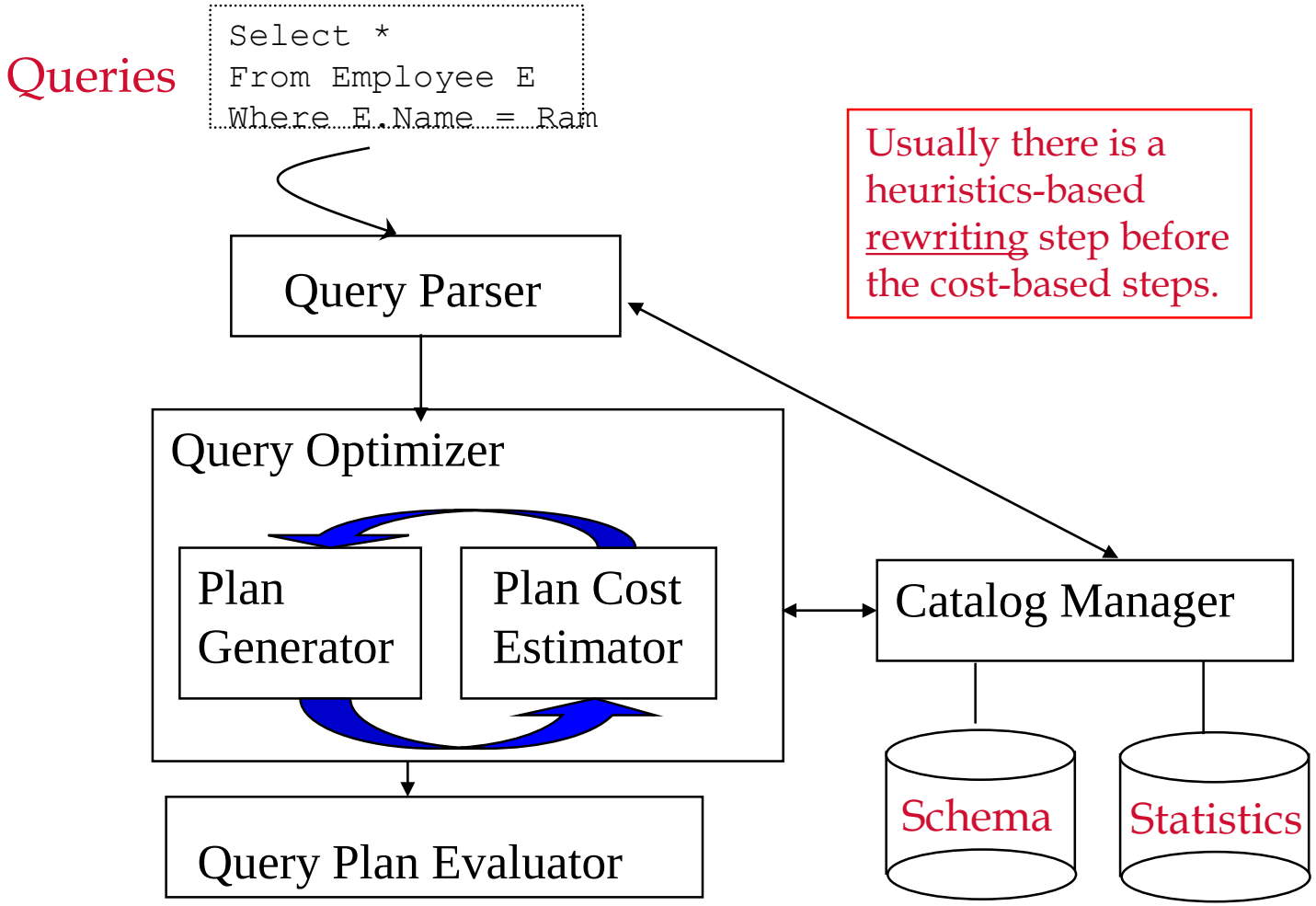
- **Incremental View Maintenance**: AVG Aggregation
  - Average cannot be maintained directly.
  - It is Maintained as: AVG = SUM/COUNT
  - Therefore both SUM and COUNT must be updated.
  - Sum = sum + value
  - Count = count - 1
  - AVG = SUM/COUNT

Cont'd…

- **Incremental View Maintenance**: Max / Min Aggregation
- While creating and updating the view maintain the Group, Max, and Count
- On Insert in the base table then update the view as:
  - If New value > current Max then update Max.
  - Otherwise only update Count.
- On delete in the base table then update the view as:
  - If deleted value is not Max then only decrement Count.
  - If deleted value is Max then recompute new Max from the group.
- Similar, reverse logic can be used for MIN aggregation maintenance.

3.3 Use of Selectivities in Cost-Based Optimization

- **Cost-based query optimization approach:**



Cont'd…

- **Cost-based query optimization approach:** A query optimizer does not depend solely on heuristic rules or query transformations; it also estimates and compares the costs of executing a query using different execution strategies and algorithms, and it then chooses the strategy with the lowest cost estimate.
  - Search the solution space to a problem for a solution that minimizes an objective (cost) function.
  - The cost functions used are estimates and not exact cost functions, so the optimization may select a query execution strategy that is not the optimal (absolute best) one.
  - Various components and information needed in cost functions to determine the query execution cost.
  - This information is kept in the DBMS Catalog. In addition DBMS uses histograms to keep details on the value distribution of important attributes.

Cont'd…

- For this approach to work, accurate cost estimates are required so that different strategies can be compared fairly and realistically.
- In addition, the optimizer must limit the number of execution strategies to be considered; otherwise, too much time will be spent making cost estimates for the many possible execution strategies.
- More suitable for compiled queries, rather than interpreted queries

Cont'd…

- **Cost components for query execution:** The cost of executing a query includes the following components:
  - Access cost to secondary storage
  - Disk storage cost
  - Computation cost
  - Memory usage cost
  - Communication cost

Cont'd…

- **Catalog Information Used in Cost Functions:** Stored in DBMS catalog and used by the query optimizer to estimate the costs of various execution strategies.
  - File size: number of records (tuples) (r), record size (R), number of file blocks (b), blocking factor (bfr)
  - File Organization: ordered, unordered, indexed, etc.
  - Index type, index attribute,
  - Number of levels (x) of each multilevel index
  - Number of first-level index blocks (b_{I1})
  - Number of distinct values of an attribute - NDV (A, R)
  - Attribute Selectivity (sl)

Cont'd…

- **Attribute selectivity(sl):**
  - Fraction of records satisfying an equality condition on the attribute.
  - Allows calculation of selection cardinality (s = sl*r)
  - Average number of records that satisfy equality selection condition on that attribute
- Some database statistics, such as index levels, change rarely and are easy to maintain, while others like the number of records change frequently.
- Query optimizers do not require perfectly up-to-date values but rely on reasonably accurate estimates to evaluate execution strategies.
- To improve result size estimation, DBMSs use histograms to represent data distribution more effectively.

Cont'd…

- **Histograms:** A histogram is a data structure maintained by the DBMS that records the distribution of attribute values.
- It helps the optimizer to estimate selectivity, result size, and query execution cost more accurately.

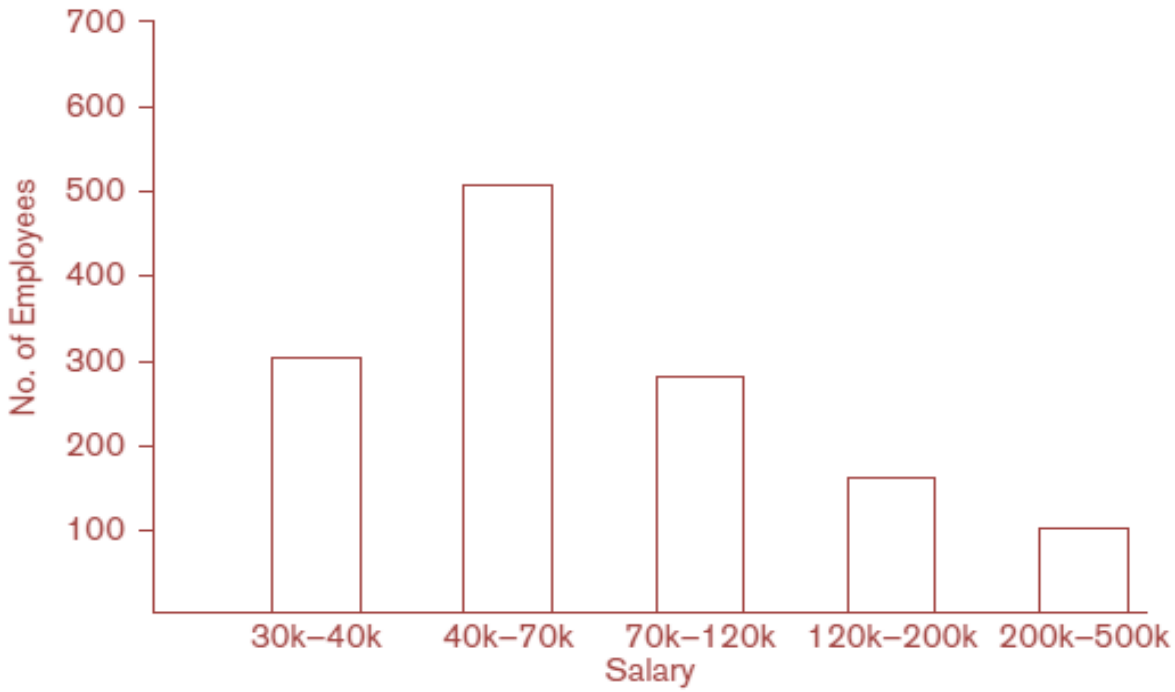


Figure: Histogram of salary in the relation EMPLOYEE

3.4 Cost Functions for SELECT Operation

- **Notation used in cost formulas:**
  - C_{si}: Cost for method si in block accesses.
  - r_x: Number of records(tuples) in a relation X
  - b_x: Number of blocks occupied by relation X (also referred to as b)
  - bfr_x: blocking factor (i.e., number of records per block) in relation X.
  - sl_A: selectivity of an attribute A for a given condition
  - s_A: Selection cardinality of the attribute being selected (= sl_A*r)
  - x_A: Number of levels of the index for attribute A
  - b_{11A}: Number of first-level blocks of the index on Attribute A.
  - NDV(A, X): Number of distinct value of attribute A in relation X.
- **Cost function**: number of block transfers between memory and disk

Cont'd…

- S1: Linear search (brute force approach)
  - Search all file blocks to retrieve all records

    $C_{S1a}=b$

  - For equality condition on a key attribute, on average one-half the records are searched

    $C_{S1b}=\frac{b}{2}$

- S2: Binary search

  $C_{S2}=\log_2 b+[\frac{s}{bfr}]-1$

  - Reduces to $\log_2 b$ if equality condition is on a key attribute

Cont'd…

- S3a: Using a primary index to retrieve a single record

  $C_{S3a} = x + 1$

- S3b: Using a hash key to retrieve a single record

  $C_{S3b} = 1$

- S4: Using an ordering index to retrieve multiple records

  $C_{S4} = x + \frac{b}{2}$

- S5: Using a clustering index to retrieve multiple records

  $C_{S5} = x + \frac{s}{bfr}$

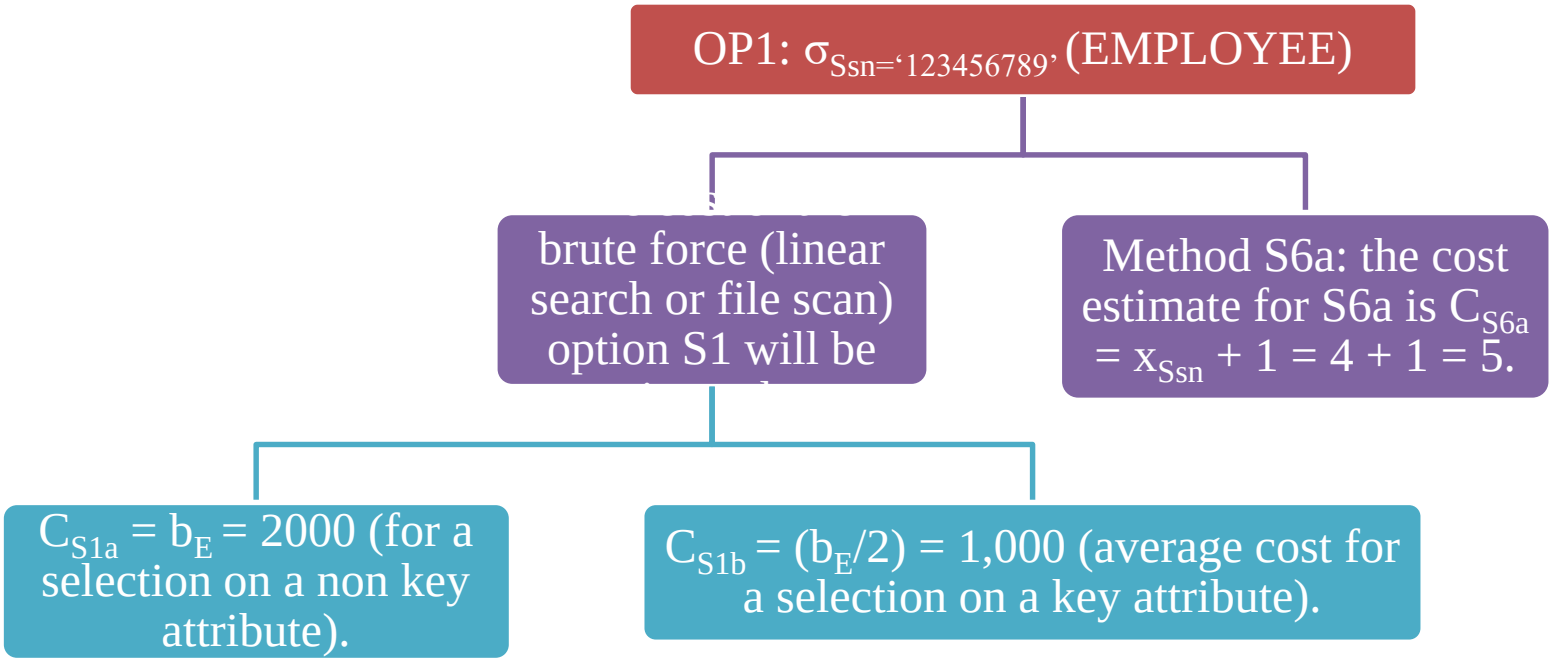
- S6: Using a secondary (B+ tree) index

  Worst Case: $C_{S6a} = x + 1 + s$

  For range queries: $C_{S4} = x + \frac{bI1}{2} + \frac{r}{2}$

Cont'd…

- **Example of Optimization of Selection Based on Cost Formulas:** Suppose that the EMPLOYEE file has $r_E = 10{,}000$ records stored in $b_E = 2{,}000$ disk blocks with blocking factor $bfr_E = 5$ records/block and the following access paths: A secondary index on the key attribute Ssn, with $x_{Ssn} = 4$.



- **Hence, Method S6a is chosen by the query optimizer over method S1.**

3.5 Cost Functions for the JOIN Operation

- Cost functions involve estimate of file size (number of tuples) that results from the JOIN operation.
- **Join selectivity**:

  $$js = 1/ \max (NDV (A, R), NDV (B,S))$$

  $$0 \leq js \leq 1$$

- **Join cardinality**: is the estimate of size of the resulting file after the join operation.

  $$jc = |(R \bowtie c\ S)| = js * |R| * |S|$$

Cont'd…

- **Assumptions:**
  - The join operations are of the form $R \bowtie_{A=B} S$
    - where A and B are domain-compatible attributes of R and S, respectively.
  - R has b_R blocks and that S has b_S blocks
  - Blocking factor for the resulting file is bfr_{RS}
  - The join selectivity (js) is known.

Cont'd…

- **J1: Nested-loop join:**
  - Assume that R is used for the outer loop.
  - For three memory buffer blocks:

    Join Cardinality of J1 (C_{J1}) = (Cost of reading blocks of R from disk to memory) + (Cost of reading the blocks of S from disk to memory) + (cost of writing the resulting file to disk).

    i.e., $C_{J1} = b_R + (b_R * b_S) + ((js * |R| * |S|) / bfr_{RS})$

  - For n_B memory buffer blocks:

    $C_{J1} = b_R + (\lceil b_R / (n_B - 2) \rceil * b_S) + ((js * |R| * |S|) / bfr_{RS})$

Cont'd…

- **J2: Index-based nested-loop join**
  - For a secondary index with selection cardinality S_B for join attribute B of S:

    $C_{J2a} = b_R + (|R|* (x_B +1+ s_B) +((js * |R| * |S|) / bfr_{RS})$
  - For a clustering index where s_B is the selection cardinality of B:

    $C_{J2b} = b_R + (|R|* (x_B + (s_B/ bfr_{RS}) +((js * |R| * |S|) / bfr_{RS})$
  - For a primary index:

    $C_{J2c} = b_R + (|R| * (x_B + 1)) + ((js * |R| * |S|)/bfr_{RS})$
  - If a hash key exists for one of the two join attributes say, B of S:

    $C_{J2d} = b_R + (|R| * h) + ((js * |R| * |S|)/bfr_{RS})$

Cont'd…

- **J3: Sort-merge join:**
  - For files already sorted on the join attributes

    $C_{J3} = b_R + b_S + ((js * |R| * |S|)/bfr_{RS})$
  - Cost of sorting must be added if sorting needed

- **J4: Partition-hash join:**

  $C_{J4} = 3 * (b_R + b_S) + ((js * |R| * |S|)/bfr_{RS})$

Cont'd…

- **3.5.1. Join selectivity and cardinality for semi-join**:
  - Join selectivity: js = MIN(1,NDV(Y, T2)/NDV(X, T1))
  - Join cardinality: jc = |T1|* js

- **Join selectivity and cardinality for anti-join**:
  - Join selectivity: js = 1- MIN(1,NDV(Y, T2)/NDV(X, T1))
  - Join cardinality: jc = |T1|* js

Cont'd…

- **3.5.2. Example of Join Optimization Based on Cost Formulas:** Assume the following statistics about the EMPLOYEE and DEPARTMENT relations:

Parameter	Description	Value
r_E	Number of records in EMPLOYEE	10,000
b_E	Number of disk blocks for EMPLOYEE	2,000
bfr_E	Blocking factor of EMPLOYEE	5 records/block
x_{Ssn}	Levels of secondary index on EMPLOYEE.Ssn	4
r_D	Number of records in DEPARTMENT	125
b_D	Number of disk blocks for DEPARTMENT	13
$x_{Dnumber}$	Levels of primary index on DEPARTMENT.Dnumber	1
s_{Mgr_ssn}	Selection cardinality on DEPARTMENT.Mgr_ssn	1
x_{Mgr_ssn}	Levels of secondary index on DEPARTMENT.Mgr_ssn	2
js_{OP6}	Join selectivity for join operation (js_{OP6})	(1/125)
bfr_{ED}	Blocking factor of resulting join file	4 records/block

Cont'd…

- Now, estimating the worst-case costs for the JOIN operation **EMPLOYEE** $\bowtie_{\text{Dno=Dnumber}}$ **DEPARTMENT** using the applicable methods as follows:

1. Using method J1 with EMPLOYEE as outer loop:

$$C_{J1} = b_E + (b_E * b_D) + ((js_{OP6} * r_E * r_D)/bfr_{ED})$$
$$= 2{,}000 + (2{,}000 * 13) + (((1/125) * 10{,}000 * 125)/4) = 30{,}500$$

2. Using method J1 with DEPARTMENT as outer loop:

$$C_{J1} = b_D + (b_E * b_D) + ((js_{OP6} * r_E * r_D)/bfr_{ED})$$
$$= 13 + (13 * 2{,}000) + (((1/125) * 10{,}000 * 125/4) = 28{,}513$$

3. Using method J2 with EMPLOYEE as outer loop:

$$C_{J2c} = b_E + (r_E * (x_{\text{Dnumber}} + 1)) + ((js_{OP6} * r_E * r_D)/bfr_{ED}$$
$$= 2{,}000 + (10{,}000 * 2) + (((1/125) * 10{,}000 * 125/4) = 24{,}500$$

4. Using method J2 with DEPARTMENT as outer loop:

$$C_{J2a} = b_D + (r_D * (x_{\text{Dno}} + s_{\text{Dno}})) + ((js_{OP6} * r_E * r_D)/bfr_{ED})$$
$$= 13 + (125 * (2 + 80)) + (((1/125) * 10{,}000 * 125/4) = 12{,}763$$

5. Using method J4 gives:

$$C_{J4} = 3* (b_D + b_E) + ((js_{OP6} * r_E * r_D)/bfr_{ED})$$
$$= 3* (13+2{,}000) + 2{,}500 = 8{,}539$$

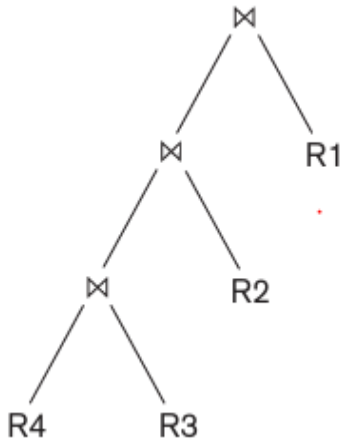
Case 5 has the lowest cost estimate and will be chosen!!!

Cont'd…

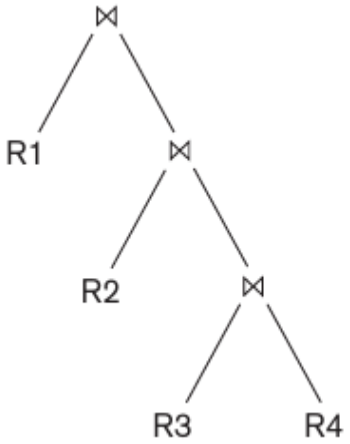
- **3.5.3. Multi-relation queries and JOIN ordering choices:**
  - In general, for a query block that has n relations, there are n! join orders.
  - Estimating the cost of every possible join tree for a query with a large number of joins will require a substantial amount of time by the query optimizer.
  - Hence, some pruning of the possible query trees is needed.
  - Query optimizers typically limit the structure of a (join) query tree to that of left-deep (or right-deep) trees.
  - The optimizer would choose the particular left-deep join tree with the lowest estimated cost.

Cont'd…

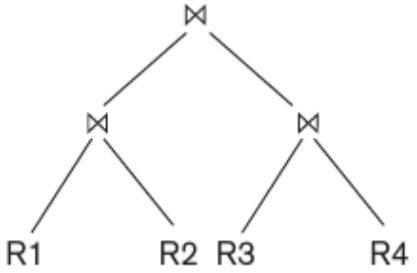
- Three options of the tree to the query optimizer:



Left-deep join tree



Right-deep join tree



Bushy join tree

- The left-deep tree has exactly one shape, and the join orders for N tables in a left-deep tree are N!.
- In Contrast, shape of the bushy tree are: $S(1) = 1.$ $S(n) = f(x) = \sum_{i=1}^{n-1} S(i) * S(n-i)$
- And the join orders of a bushy tree are $(2n-2)!/(n-1)!$

Cont'd…

- For example, number of permutations of left-deep and bushy join trees of n relations is shown in table below:

No. of Relations N	No. of Left Deep Trees N!	No. of Bushy Shapes S(N)	No. of Bushy Trees (2N-2)!/(N-1)!
2	2	1	2
3	6	2	12
4	24	5	120
5	120	14	1680
6	720	42	30240
7	5040	132	665280

Cont'd…

- **Advantage** of left-deep trees over the bushy trees includes the following:
  - Allow to generate fully pipelined plans
  - Work well for common algorithms for join
- Hence, left-deep trees are preferred than bushy join trees.
- However, in case of materialization (instead of pipelining) for the join ordering is to find an ordering that will reduce the size of the temporary results.

Cont'd…

- **3.5.4. Physical Optimization:**
  - In addition to optimizing logical query plan based on the heuristics, physical optimization is also required.
  - In physical optimization, for each operation a further decision in terms of executing the operation by a specific algorithm at the physical level is made.
  - If this optimization is based on the relative cost of each possible implementation, it is called cost-based physical optimization.
  - The two sets of approaches to this decision making :
    - Top-down approach and Bottom-up approach : Common in RDBMS as it supports pipelining
  - If the number of relations is small and possible implementations options are limited, then most optimizers would elect to apply a cost-based optimization approach directly rather than to explore heuristics.
  - However, certain physical level heuristics make cost based optimizations unnecessary

Cont'd…

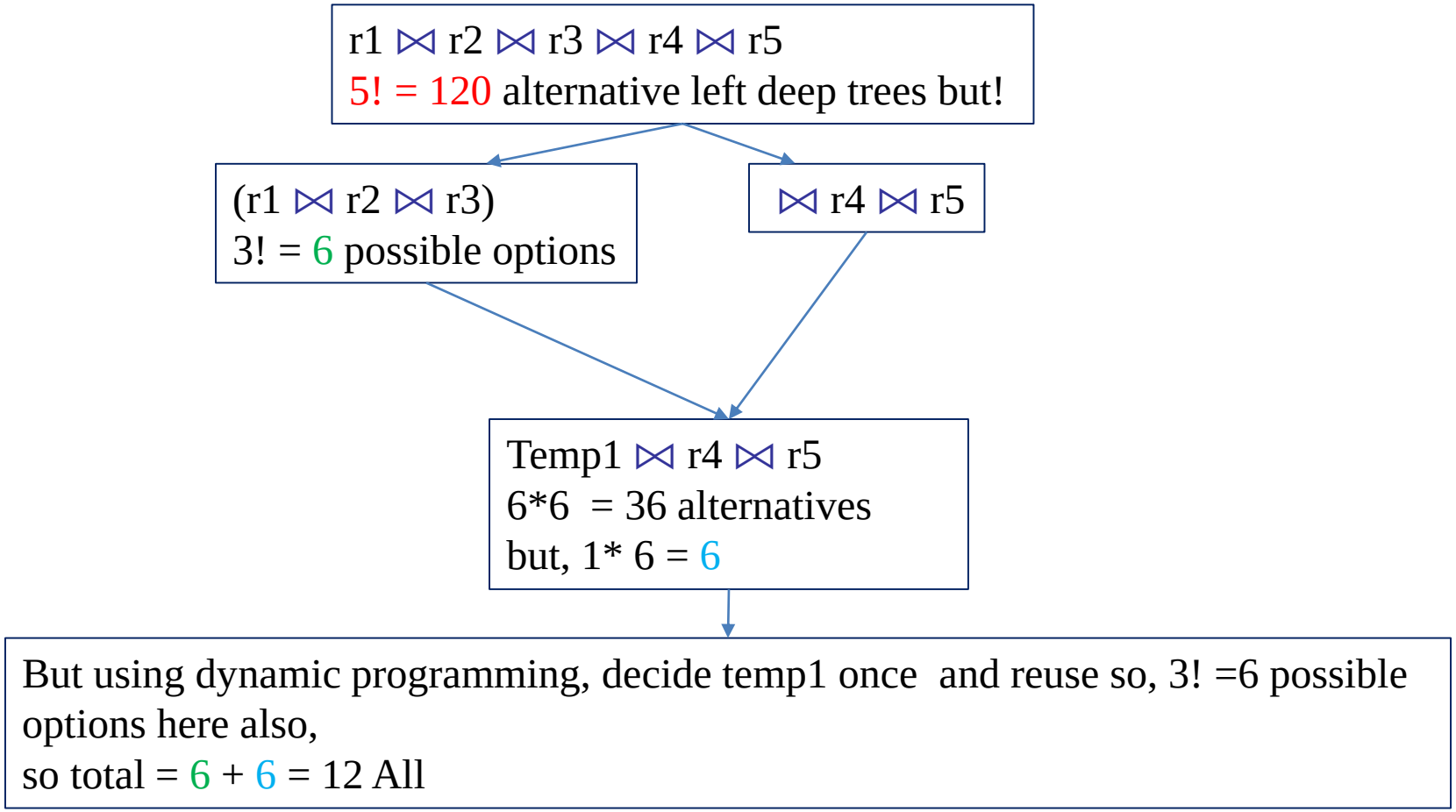
- **These heuristics include:**
  - For selections, use index scans wherever possible.
  - If the selection condition is conjunctive, use the selection that results in the smallest cardinality first.
  - If the relations are already sorted on the attributes being matched in a join, then prefer sort-merge join to other join methods.
  - For union and intersection of more than two relations, use the associative rule; consider the relations in the ascending order of their estimated cardinalities.
  - If one of the arguments in a join has an index on the join attribute, use that as the inner relation.
  - If the left relation is small and the right relation is large and it has index on the joining column, then try index-based nested-loop join.
  - Consider only those join orders where there are no Cartesian products or where all joins appear before Cartesian products.

Cont'd…

- **3.5.5. Dynamic Programming Approach to Join Ordering:**
  - Dynamic programming (a greedy heuristic approach) is an optimization technique where subproblems are solved only once, and it is applicable when a problem may be broken down into subproblems that themselves have subproblems.
  - A typical dynamic programming algorithm has the following characteristics:
    - Optimal solution structure is developed
    - Value of the optimal solution is recursively defined
    - Optimal solution is computed and its value developed in a bottom-up fashion
  - Dynamic programming may be applied to the join order selection i.e., evaluate possible permutations of joining relations

Cont'd…

- **Dynamic programming to the join order selection**: consider the problem of ordering a 5-way join of relations r1, r2, r3, r4, r5.



3.6 Example to Illustrate Cost-Based Query Optimization

- Consider following query Q2:

```
SELECT Pnumber, Dnum, Lname, Address, Bdate
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum=Dnumber AND Mgr_ssn=Ssn AND Plocation='Stafford';
```

- Now, Evaluating potential join orders:

PROJECT ⋈ DEPARTMENT ⋈ EMPLOYEE

DEPARTMENT ⋈ PROJECT ⋈ EMPLOYEE

DEPARTMENT ⋈ EMPLOYEE ⋈ PROJECT

EMPLOYEE ⋈ DEPARTMENT ⋈ PROJECT

Cont'd…

- Figure below shows sample statistical information for relations in Q2. (a) Column information (b) Table information (c) Index information

(a)

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

(b)

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000

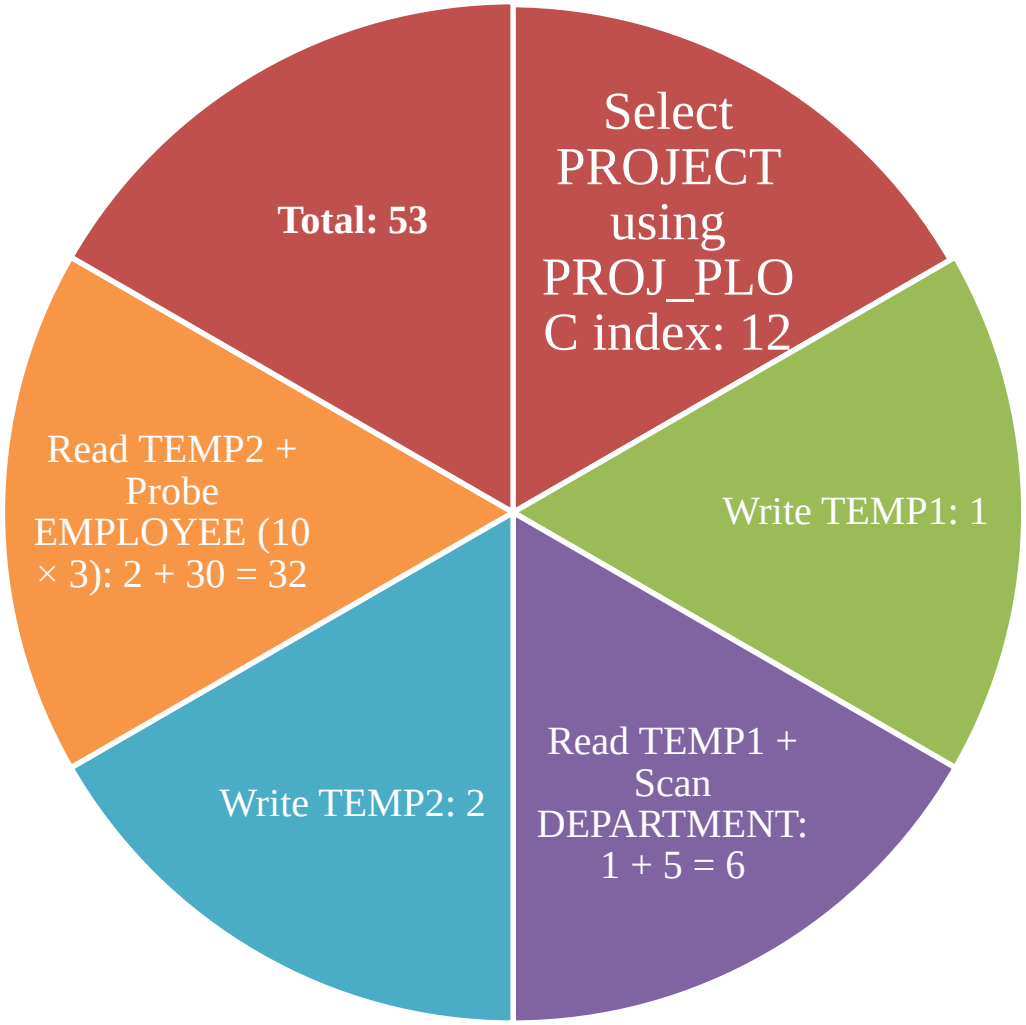
(c)

Index_name	Uniqueness	Blevel*	Leaf_blocks	Distinct_keys
PROJ_PLOC	NONUNIQUE	1	4	200
EMP_SSN	UNIQUE	1	50	10000
EMP_SAL	NONUNIQUE	1	50	500

*Blevel is the number of levels without the leaf level.

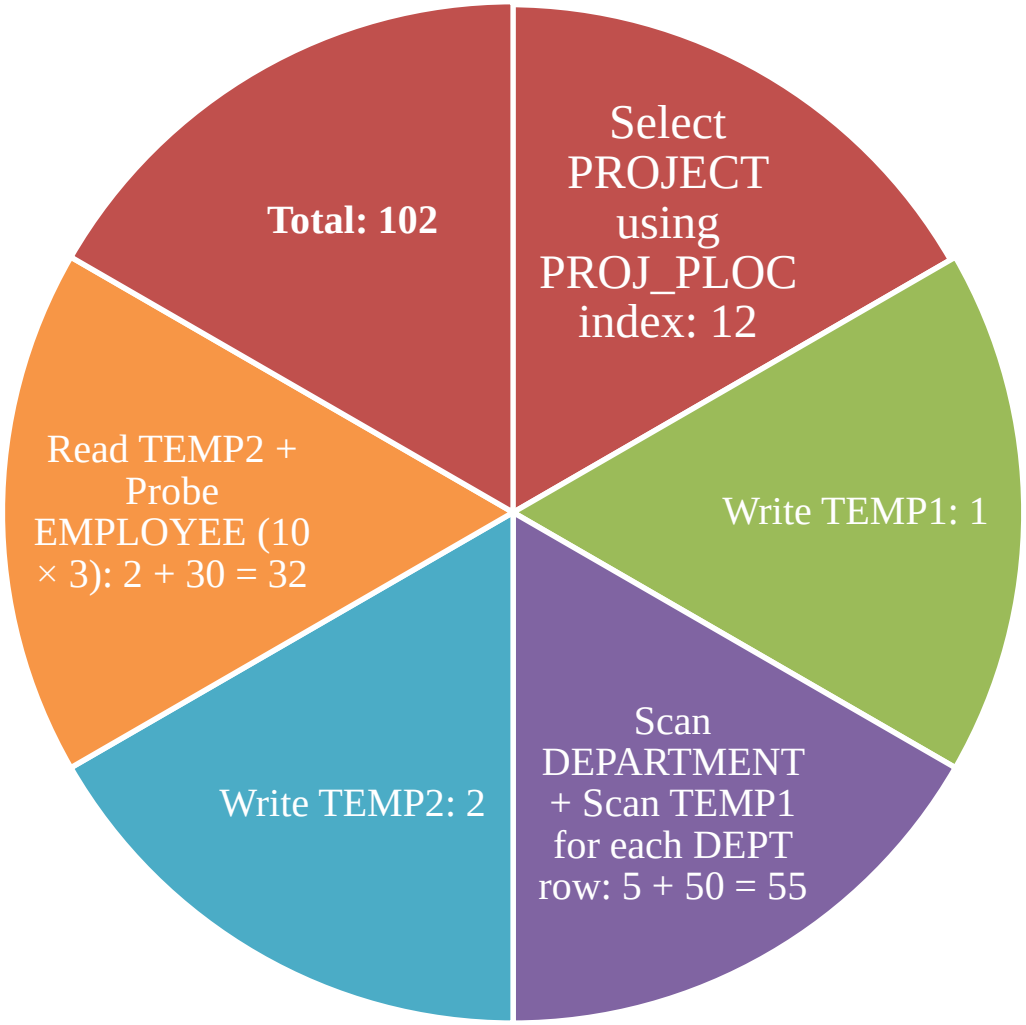
Cont'd…

- **Order1: PROJECT ⋈ DEPARTMENT ⋈ EMPLOYEE:**



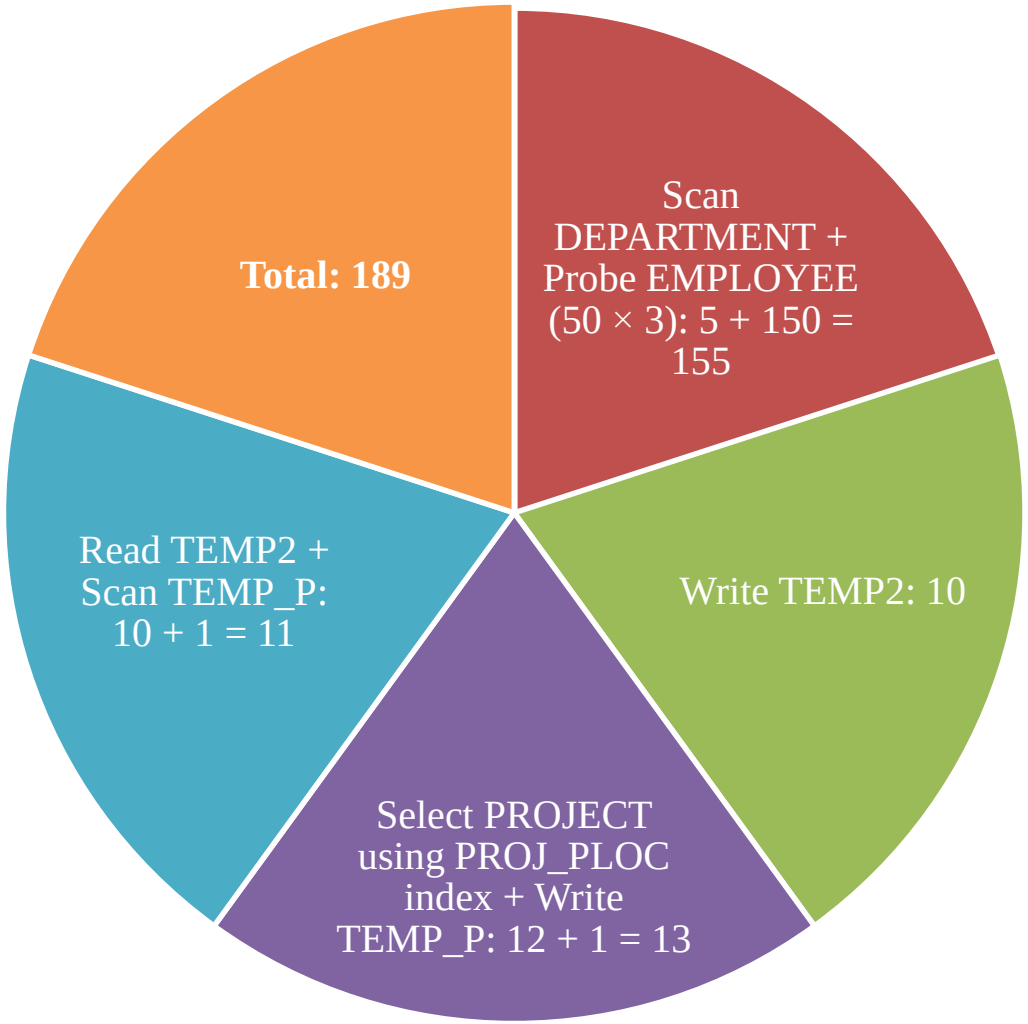
Cont'd…

- **Order2: DEPARTMENT ⋈ PROJECT ⋈ EMPLOYEE**



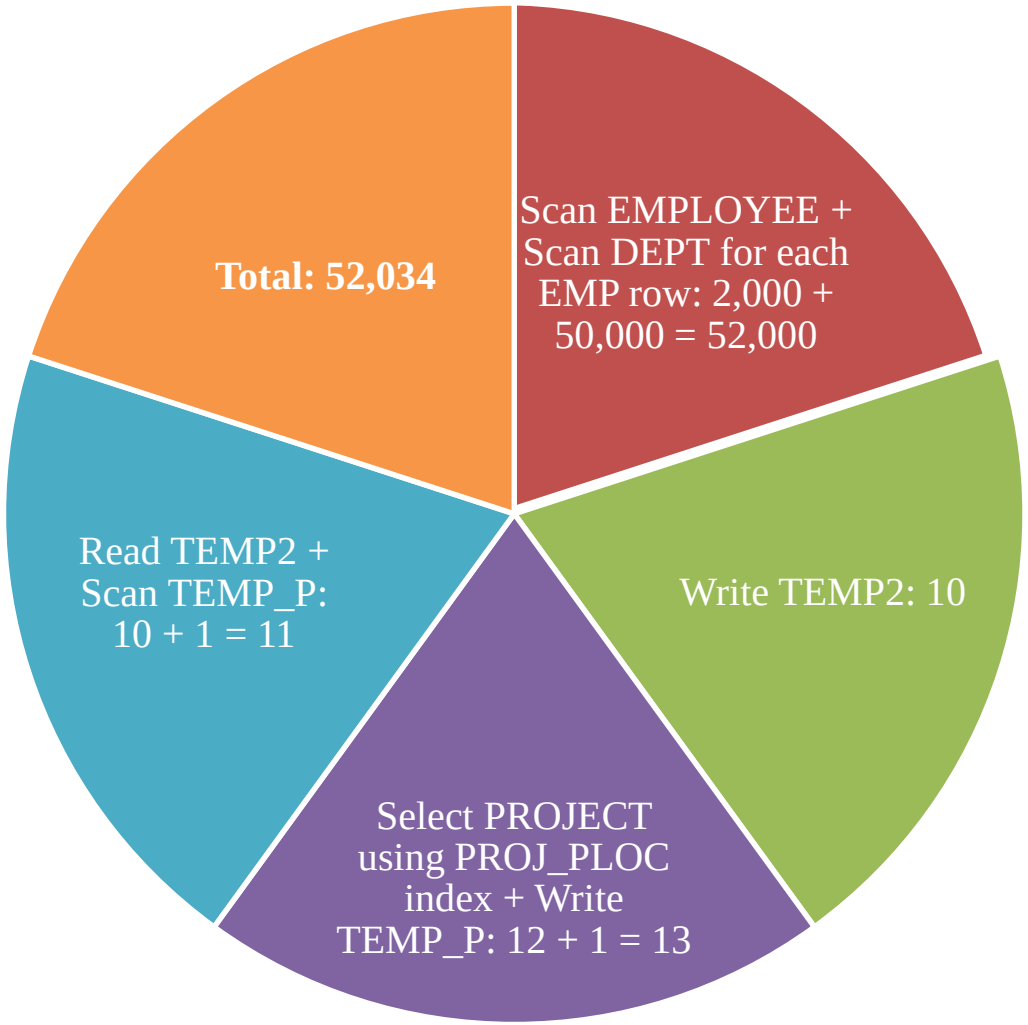
Cont'd…

- **Order3: DEPARTMENT ⋈ EMPLOYEE ⋈ PROJECT:**



Cont'd…

- **Order 4: EMPLOYEE ⋈ DEPARTMENT ⋈ PROJECT**



Cont'd…

- Hence, the Join order 1 looks cost effecting in this database scenario hence selected by the query optimizer as its query execution plan.

Order1:

PROJECT ⋈ DEPARTMENT ⋈ EMPLOYEE

Cost = ~53 bock accesses, hence best

Order2:

DEPARTMENT ⋈ PROJECT ⋈ EMPLOYEE

Cost = ~102 bock accesses

Order3:

DEPARTMENT ⋈ EMPLOYEE ⋈ PROJECT

Cost = ~189 bock accesses

Order4:

EMPLOYEE ⋈ DEPARTMENT ⋈ PROJECT

Cost = ~52034 bock accesses

3.7 Additional Issues Related to Query Optimization

- **Displaying the system's query execution plan**
  - Oracle syntax
    - EXPLAIN PLAN FOR <SQL query>
  - IBM DB2 syntax
    - EXPLAIN PLAN SELECTION [additional options] FOR <SQL-query>
  - SQL server syntax
    - SET SHOWPLAN_TEXT ON or SET SHOWPLAN_XML ON or SET SHOWPLAN_ALL ON
  - PostgreSQL:
    - EXPLAIN [set of options] .where the options include ANALYZE, VERBOSE, COSTS, BUFFERS, TIMING, etc.

Cont'd…

- **Size estimation of other operations**: Projection
  - For projection of the form πList (R) expressed as SELECT <attribute list> FROM R, since SQL treats it as a multiset, the estimated number of tuples in the result is |R|.
  - If the DISTINCT option is used, then size of π_A (R) is NDV (A, R).

Cont'd…

- **Size estimation of other operations**: Set operations
  - If the arguments for an intersection, union, or set difference are made of selections on the same relation, they can be rewritten as conjunction, disjunction, or negation, respectively.
  - For example, $\sigma_{c1}(R) \cap \sigma_{c2}(R)$ can be rewritten as $\sigma_{c1 \text{ AND } c2}(R)$; and $\sigma_{c1}(R) \cup \sigma_{c2}(R)$ can be rewritten as $\sigma_{c1 \text{ OR } c2}(R)$.
  - The size estimation can be made based on the selectivity of conditions c1 and c2.
  - Otherwise, the estimated upper bound on the size of $r \cap s$ is the minimum of the sizes of r and s; the estimated upper bound on the size of $r \cup s$ is the sum of their sizes.

Cont'd…

- **Size estimation of other operations**: Aggregation
  - The size of $G\Im_{\text{Aggregate-function}}(A)$ R is NDV (G, R) since there is one group for each unique value of G.

Cont'd…

- **Size estimation of other operations**: Outer join
  - The size of R LEFT OUTER JOIN S would be |R ⋈ S| plus |R anti-join S|.
  - Similarly, the size of R FULL OUTER JOIN S would be |r ⋈ s| plus |r anti-join s| plus |s anti-join r|.

Cont'd…

- **Plan caching:**
  - Plan stored by query optimizer for later use by same queries with different parameters.
  - For example, a bank teller uses an account number and some function code to check the balance in that account.
  - To run such queries or transactions repeatedly, the query optimizer computes the best plan when the query is submitted for the first time and caches the plan for future use.
  - This storing of the plan and reusing it is referred to as plan caching.
  - When the query is resubmitted with different constants as parameters, the same plan is reused with the new parameters.

Cont'd…

- **Parametric query Optimization**
  - It is conceivable that the plan may need to be modified under certain situations;
    - for example, if the query involves report generation over a range of dates or range of accounts, then, depending on the amount of data involved, different strategies may apply.
  - Under a variation called parametric query optimization, a query is optimized without a certain set of values for its parameters and the optimizer outputs a number of plans for different possible value sets, all of which are cached.
  - As a query is submitted, the parameters are compared to the ones used for the various plans and the cheapest among the applicable plans is used.

Cont'd…

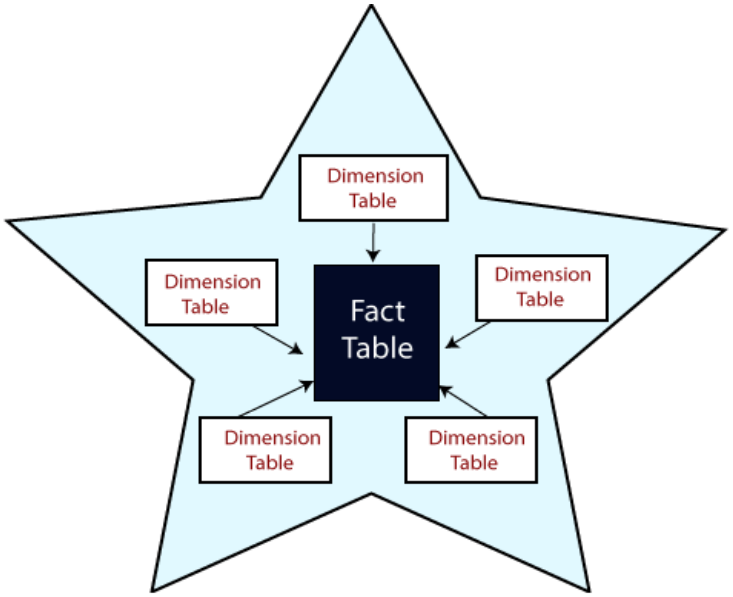
- **Top k-results optimization:** Limits strategy generation
  - When the output of a query is expected to be large, sometimes the user is satisfied with only the top-k results based on some sort order.
  - Some RDBMSs have a limit K clause to limit the result to that size.
  - DBMS may uses strategy of generation of results in a sorted order so that it can be stopped after K tuples.

3.8 An Example of Query Optimization in Data Warehouses

- Query optimization in data warehouses often focuses on star schema queries, where a large fact table is joined with multiple smaller dimension tables.
- These queries are common in analytical systems and involve:
  - Filtering dimension tables
  - Joining them with a large fact table
  - Performing aggregation (e.g., SUM, COUNT)

Cont'd…

- **Star Schema Structure:** A star schema consists of:
  - Central fact table
    - Very large table
    - Stores measures such as sales amount, quantity
  - Dimension tables surrounds the fact table.
    - Small tables such as customer, product, time
    - Contain descriptive attributes
  - The fact table links to dimension tables via foreign keys.



Star Schema

Cont'd…

- **Star Transformation Optimization:**
  - The main objective is Reduce access to the fact table as much as possible, avoiding a full table scan.
  - This is achieved by filtering dimension tables first and using them to limit fact table rows.
  - Two types of Star Transformation Optimization:
    - Classic Star Transformation
    - Bitmap Index Star Transformation

Cont'd…

- **Classic Star Transformation**
  - Apply filters on dimension tables first
  - Perform Cartesian product of filtered dimension results
  - Join result with fact table using indexes usually B-tree indexes.
  - Hence, reduces fact table search space before joining.

Cont'd…

- **Bitmap Index Star Transformation** (More Efficient)
  - Requires bitmap indexes on fact table foreign keys.
  - It Involves the following Steps:
    - Apply filters on dimension tables
    - Get matching primary keys
    - Use bitmap indexes on fact table foreign keys
    - Convert matching rows into bitmaps
    - Combine bitmaps:
      - OR within each dimension
      - AND across dimensions
  - Retrieve only matching fact table rows using tuple IDs

Cont'd…

Example Query:

```
SELECT D1.X, D2.Y, SUM(F.M)
FROM FACT F, DIM1 D1, DIM2 D2, DIM3 D3
WHERE F.Fk1 = D1.Pk
  AND F.Fk2 = D2.Pk
  AND F.Fk3 = D3.Pk
  AND D1.A > 5
  AND D2.B < 77
  AND D3.C = 11
GROUP BY D1.X, D2.Y;
```

Transformed Query: The optimizer rewrites the query using subqueries:

```
F.Fk1 IN (SELECT D1.Pk FROM DIM1 WHERE D1.A > 5)
F.Fk2 IN (SELECT D2.Pk FROM DIM2 WHERE D2.B < 77)
F.Fk3 IN (SELECT D3.Pk FROM DIM3 WHERE D3.C = 11)
```

Cont'd…

- **Joining Back:**
  - Sometimes dimension tables must be joined again after filtering.
  - But join-back is not needed if:
    - Dimension key is unique (primary key)
    - No dimension columns are used in SELECT or GROUP BY
    - Example: DIM3 is not re-joined because it is only used for filtering

Thank You !